



Welcome to this compre...
TXT 301.59KB

make notes for me out of this transcript , give proper code examples
with explanations

Vue.js Complete Course Notes

Table of Contents

1. Introduction to Vue.js
2. Vue Fundamentals
3. Vue Directives
4. Event Handling
5. Data Binding with v-model
6. Computed Properties & Watchers
7. Components
8. Component Communication
9. Slots
10. Lifecycle Hooks
11. Composables
12. Vue Router
13. API Requests with Axios
14. Pinia State Management

1. Introduction to Vue.js

What is Vue.js?

- Progressive JavaScript framework for building user interfaces and SPAs
- Created by Evan You, released in 2014
- Uses pure HTML-based templates (unlike React's JSX or Angular's TypeScript)

- Gentle learning curve, smaller bundle size, highly flexible

Why Learn Vue?

- **Easy to learn:** If you know HTML, CSS, and JavaScript, you're most of the way there
- **Reactive:** UI automatically updates when data changes
- **Component-based:** Build reusable components
- **Great ecosystem:** Vue CLI, Vite, Vue Router, Pinia, and many third-party plugins

Vue 3 vs Options API vs Composition API

Feature	Options API	Composition API
Learning curve	Easier	Slightly steeper
Organization	By option type (data, methods, computed)	By logical concern
Reusability	Mixins	Composables
Bundle size	Larger	Smaller
TypeScript support	Limited	Excellent

This course uses Vue 3 with Composition API.

2. Vue Fundamentals

Getting Started with CDN

```
html

<!DOCTYPE html>
<html>
<head>
  <title>Vue Fundamentals</title>
  <!-- Vue CDN -->
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
```

```
<body>
  <div id="app">
    <h1>{{ message }}</h1>
    <p>Lucky number: {{ luckyNumber }}</p>
  </div>

  <script>
    const { createApp, ref, reactive, computed, watch } = Vue;

    const app = createApp({
      setup() {
        // Primitive reactive value
        const message = ref('Hello Vue 3');
        const luckyNumber = ref(6);

        // Complex reactive object
        const employee = reactive({
          name: 'John',
          age: 30,
          salary: 50000
        });

        // Array
        const luckyNumbers = ref([1, 2, 3, 4, 5]);

        return {
          message,
          luckyNumber,
          employee,
          luckyNumbers
        };
      }
    });

    app.mount('#app');
  </script>
</body>
</html>
```

Key Concepts

ref() VS reactive()

javascript

```
import { ref, reactive } from 'vue';

// ref - for primitives (string, number, boolean)
const count = ref(0);
console.log(count.value); // Access with .value
count.value++; // Update with .value

// reactive - for objects/arrays
const state = reactive({
  name: 'John',
  age: 30
});
console.log(state.name); // No .value needed
state.age = 31; // Direct update
```

Rules:

- Use `ref` for primitive types (string, number, boolean)
- Use `reactive` for objects and arrays
- `ref` can also hold objects, but requires `.value` for updates
- `ref` calls `reactive` behind the scenes

Template Interpolation

html

```
<!-- String interpolation -->
<h1>{{ message }}</h1>

<!-- JavaScript expressions -->
<p>{{ luckyNumber % 2 === 0 ? 'Even' : 'Odd' }}</p>
<p>{{ Math.floor(Math.random() * 10) + 1 }}</p>

<!-- Accessing object properties -->
<p>{{ employee.name }} is {{ employee.age }} years old</p>

<!-- Array access -->
<p>{{ luckyNumbers[0] }}</p>
```

3. Vue Directives

Directives are special attributes that start with `v-` and apply reactive behavior to the DOM.

v-html

Renders HTML content dynamically.

```
html

<template>
  <span v-html="getMessage()"></span>
</template>

<script setup>
const message = 'Hello Vue';

function getMessage() {
  return `<h1><u>${message}</u></h1>`;
}
</script>
```

v-once

Renders the element only once; subsequent updates are ignored.

```
html

<p v-once>Render once: {{ luckyNumber }}</p>
<p>Regular: {{ luckyNumber }}</p>
```

v-bind (Shorthand: :)

Binds HTML attributes to Vue data.

```
html

<!-- Full syntax -->
<a v-bind:href="blogUrl">Visit Blog</a>

<!-- Shorthand -->
<a :href="blogUrl">Visit Blog</a>

<!-- Binding multiple attributes -->
```

```



<!-- Class binding -->
<div :class="{ active: isActive, 'text-danger': hasError }"></div>

<!-- Style binding -->
<div :style="{ backgroundColor: bgColor, fontSize: fontSize + 'px' }"></div>

<!-- Array syntax for classes -->
<div :class="[baseClass, { active: isActive }]"></div>

```

v-if / v-else-if / v-else

Conditionally renders elements (removes from DOM when false).

html

```

<p v-if="employee.hasCorporateCard">
  Spending limit: {{ employee.spendingLimit }}
</p>
<p v-else-if="employee.salary > 25000 && employee.yearsWithCompany > 1">
  Eligible for corporate card
</p>
<p v-else>
  Not eligible for corporate card
</p>

```

v-show

Conditionally displays elements (uses display: none CSS).

html

```

<p v-show="employee.hasCorporateCard">
  Spending limit: {{ employee.spendingLimit }}
</p>

```

v-if vs v-show:

v-if

v-show

Removes/adds from DOM

Toggles display: none

v-if**v-show**

Better for infrequent toggles

Better for frequent toggles

Higher toggle cost

Higher initial render cost

Supports v-else

No v-else support

v-for

Iterates over arrays and objects.

html

```
<!-- Array iteration -->
<ul>
  <li v-for="(luckyNumber, index) in luckyNumbers" :key="index">
    {{ index + 1 }}. {{ luckyNumber }}
  </li>
</ul>

<!-- Array of objects -->
<ul>
  <li v-for="(project, index) in employee.projects" :key="project.project">
    {{ index + 1 }} - {{ project.project }} - {{ project.timeline }}
  </li>
</ul>

<!-- Object iteration -->
<ul>
  <li v-for="(value, key, index) in employee" :key="key">
    {{ key }}: {{ value }}
  </li>
</ul>
```

Key Attribute:

- Always use `:key` with `v-for` for proper DOM patching
- Use a unique identifier (ID, name, etc.)
- Critical when reordering/shuffling elements

html

```

<!-- Without key: input values don't follow their projects when shuffled -->
<!-- With key: input values stay with their projects -->
<div v-for="project in employee.projects" :key="project.project">
  <span>{{ project.project }}</span>
  <input type="text" placeholder="Comments">
</div>

```

v-model

Creates two-way data binding on form inputs.

html

```

<!-- Text input -->
<input type="text" v-model="name">

<!-- Number input -->
<input type="number" v-model.number="age">

<!-- Checkbox -->
<input type="checkbox" v-model="subscribe">

<!-- Radio buttons -->
<input type="radio" v-model="gender" value="male"> Male
<input type="radio" v-model="gender" value="female"> Female

<!-- Select dropdown -->
<select v-model="gender">
  <option value="male">Male</option>
  <option value="female">Female</option>
</select>

```

Model Modifiers:

- `.trim` - Removes whitespace
- `.lazy` - Updates on change event instead of input
- `.number` - Converts to number

html

```

<input v-model.trim="name">
<input v-model.lazy="age">
<input v-model.number="phone">

```

4. Event Handling

v-on (Shorthand: @)

Attaches event listeners.

html

```
<!-- Full syntax -->
<button v-on:click="handleClick">Click Me</button>

<!-- Shorthand -->
<button @click="handleClick">Click Me</button>

<!-- With parameters -->
<button @click="handleClick('Hello Vue')">Click Me</button>

<!-- With event object -->
<button @click="handleClick($event)">Click Me</button>

<!-- Inline handler -->
<button @click="count++">Increment</button>
```

Event Modifiers

html

```
<!-- Stop propagation -->
<button @click.stop="handleClick">Click</button>

<!-- Prevent default -->
<form @submit.prevent="handleSubmit">
  <button type="submit">Submit</button>
</form>

<!-- Only trigger once -->
<button @click.once="handleClick">Click Once</button>

<!-- Key modifiers -->
<input @keyup.enter="handleEnter">
<input @keyup.ctrl.enter="handleCtrlEnter">

<!-- Mouse button modifiers -->
```

```
<button @click.left="handleLeftClick">Left Click</button>
<button @click.right="handleRightClick">Right Click</button>
```

Event Object

javascript

```
function handleClick(event) {
  console.log(event.type); // "click"
  console.log(event.clientX, event.clientY); // Mouse coordinates
  console.log(event.target); // The element that was clicked
}
```

5. Data Binding with v-model

Two-Way Binding

html

```
<template>
  <input v-model="name" placeholder="Enter name">
  <p>Name: {{ name }}</p>

  <input type="number" v-model.number="age">
  <p>Age: {{ age }}</p>

  <input type="checkbox" v-model="subscribe">
  <label>Subscribe to newsletter</label>

  <select v-model="gender">
    <option value="male">Male</option>
    <option value="female">Female</option>
  </select>
</template>

<script setup>
import { ref } from 'vue';

const name = ref('Ben');
const age = ref(20);
const subscribe = ref(false);
```

```
const gender = ref('male');
</script>
```

Form Validation Example

html

```
<template>
  <form @submit.prevent="handleSubmit">
    <div class="mb-3">
      <label>Name:</label>
      <input
        v-model.trim="formObj.name"
        @blur="validateName"
        class="form-control"
      >
      <div v-if="!formObj.isNameValid" class="text-danger">
        Name is required (min 3 characters)
      </div>
    </div>

    <div class="mb-3">
      <label>Email:</label>
      <input v-model.trim="formObj.email" class="form-control">
    </div>

    <div class="mb-3">
      <label>Phone:</label>
      <input v-model.number="formObj.phone" class="form-control">
    </div>

    <div class="form-check">
      <input type="checkbox" v-model="formObj.subscribe" class="form-check-input">
      <label class="form-check-label">Subscribe to newsletter</label>
    </div>

    <!-- Error List -->
    <div v-if="errorList.length > 0" class="mt-3">
      <ul class="text-danger">
        <li v-for="error in errorList" :key="error">{{ error }}</li>
      </ul>
    </div>

    <button type="submit" class="btn btn-primary mt-3">Submit</button>
  </form>
</template>
```

```
<script setup>
import { reactive } from 'vue';

const formObj = reactive({
  name: '',
  email: '',
  phone: '',
  subscribe: false,
  gender: 'male',
  isNameValid: true
});

const errorList = reactive([]);

function validateName() {
  formObj.isNameValid = formObj.name.length >= 3;
}

function handleSubmit() {
  errorList.length = 0; // Clear errors

  if (formObj.name.length < 3) {
    errorList.push('Name must be at least 3 characters');
  }
  if (!formObj.email) {
    errorList.push('Email is required');
  }
  if (!formObj.subscribe) {
    errorList.push('Please subscribe to the newsletter');
  }

  if (errorList.length === 0) {
    alert('Form submitted successfully!');
    console.log('Form data:', formObj);
  }
}
</script>
```

6. Computed Properties & Watchers

Computed Properties

Computed properties are cached based on their reactive dependencies. They only re-evaluate when dependencies change.

javascript

```
import { ref, computed } from 'vue';

const firstName = ref('Ben');
const lastName = ref('Patel');

// Computed property (cached)
const fullName = computed(() => {
  console.log('Computing full name...');
  return `${firstName.value} ${lastName.value}`;
});

// Regular function (not cached, runs on every render)
function getFullName() {
  console.log('Getting full name...');
  return `${firstName.value} ${lastName.value}`;
}
```

Accessing Previous Value:

javascript

```
const fullName = computed((previous) => {
  if (previous === 'Ben Patel') {
    console.log('Special case: Ben Patel detected');
  }
  return `${firstName.value} ${lastName.value}`;
});
```

Watchers

Watchers observe and react to changes in specific data properties.

javascript

```
import { ref, watch, reactive } from 'vue';

const counter = ref(0);
const obj = reactive({ count: 0 });

// Watch a ref
```

```
watch(counter, (newValue, oldValue) => {
  console.log(`Counter changed from ${oldValue} to ${newValue}`);
  if (newValue === 5) {
    alert('Counter reached 5!');
  }
});

// Watch a reactive property (must use getter)
watch(
  () => obj.count,
  (newValue, oldValue) => {
    console.log(`Obj count changed from ${oldValue} to ${newValue}`);
  }
);

// Async watcher
watch(counter, async (newValue) => {
  if (newValue === 5) {
    await new Promise(resolve => setTimeout(resolve, 3000));
    console.log('Async operation completed');
  }
});
```

Computed vs Watchers

Computed	Watchers
Must return a value	Not required to return
Cached	Not cached
Synchronous only	Can be asynchronous
Declarative	Imperative
For derived values	For side effects

7. Components

Creating a Vue Project with Vite

```
bash
```

```
npm create vue@latest  
cd project-name  
npm install  
npm run dev
```

Project Structure

```
text
```

```
project/  
├─ public/  
│   └─ favicon.ico  
├─ src/  
│   ├─ assets/  
│   ├─ components/  
│   │   └─ MyComponent.vue  
│   ├─ App.vue  
│   └─ main.js  
├─ index.html  
├─ package.json  
└─ vite.config.js
```

main.js

```
javascript
```

```
import { createApp } from 'vue';  
import App from './App.vue';  
  
const app = createApp(App);  
app.mount('#app');
```

App.vue

```
vue
```

```
<template>  
  <div id="app">  
    <h1>{{ message }}</h1>  
  </div>  
</template>
```

```
<script setup>
import { ref } from 'vue';

const message = ref('Hello Vue 3');
</script>

<style scoped>
h1 {
  color: green;
}
</style>
```

Creating a Component

vue

```
<!-- components/ButtonCounter.vue -->
<template>
  <button @click="count++" class="btn btn-primary">
    Clicked {{ count }} times
  </button>
</template>

<script setup>
import { ref } from 'vue';

const count = ref(0);
</script>
```

Registering Components

Global Registration (main.js):

javascript

```
import { createApp } from 'vue';
import App from './App.vue';
import ButtonCounter from './components/ButtonCounter.vue';

const app = createApp(App);

// Register globally
app.component('button-counter', ButtonCounter);
```

```
app.mount('#app');
```

Local Registration (recommended):

vue

```
<!-- App.vue -->
<template>
  <div>
    <button-counter />
  </div>
</template>

<script setup>
import ButtonCounter from './components/ButtonCounter.vue';
</script>
```

Note: With `<script setup>`, imported components are automatically available in the template.

8. Component Communication

Props (Parent to Child)

vue

```
<!-- Parent.vue -->
<template>
  <Contact
    name="Ben"
    :phone="1234567890"
    :owner-name="contactOwner"
    :is-favorite="true"
  />
</template>

<script setup>
import { ref } from 'vue';
import Contact from './components/Contact.vue';
```

```
const contactOwner = ref('NetMastery');
</script>
```

vue

```
<!-- Child.vue (Contact.vue) -->
<template>
  <div>
    <h3>{{ name }}</h3>
    <p>Phone: {{ phone }}</p>
    <p v-if="ownerName">Owner: {{ ownerName }}</p>
  </div>
</template>

<script setup>
// Define props with validation
const props = defineProps({
  name: {
    type: String,
    required: true
  },
  phone: {
    type: Number,
    required: true
  },
  ownerName: {
    type: String,
    default: 'Not Applicable'
  },
  isFavorite: {
    type: Boolean,
    default: false
  }
});
</script>
```

Important:

- Props are read-only (child cannot modify directly)
- Use `:prop-name` for dynamic values
- Use `prop-name="static"` for static strings
- Case-sensitive: use kebab-case in templates, camelCase in script

Emits (Child to Parent)

vue

```
<!-- Child.vue -->
<template>
  <button @click="toggleFavorite">Toggle Favorite</button>
</template>

<script setup>
const props = defineProps(['isFavorite']);
const emit = defineEmits(['update-favorite']);

function toggleFavorite() {
  // Emit event with payload
  emit('update-favorite', {
    isFavorite: props.isFavorite,
    name: props.name
  });
}
</script>
```

vue

```
<!-- Parent.vue -->
<template>
  <Contact
    :is-favorite="contact.isFavorite"
    @update-favorite="handleUpdateFavorite"
  />
</template>

<script setup>
function handleUpdateFavorite(eventData) {
  // eventData contains the emitted payload
  console.log(eventData);
  contact.isFavorite = !eventData.isFavorite;
}
</script>
```

Passing Methods as Props (Alternative to Emits)

vue

```

<!-- Parent.vue -->
<template>
  <AddContact :on-add-contact="handleAddContact" />
</template>

<script setup>
function handleAddContact(contact) {
  contacts.push({
    ...contact,
    ownerName: 'NetMastery',
    isFavorite: false
  });
}
</script>

```

vue

```

<!-- Child.vue (AddContact.vue) -->
<script setup>
const props = defineProps({
  onAddContact: {
    type: Function,
    required: true
  }
});

function addContact() {
  props.onAddContact({
    name: newContact.name,
    phone: newContact.phone,
    email: newContact.email
  });
}
</script>

```

Provide/Inject (Avoid Prop Drilling)

javascript

```

// Parent (App.vue)
import { provide, ref } from 'vue';

const maxNumber = ref(100);

```

```
// Provide value with a unique key
provide('max-lucky-number', maxNumber);

javascript

// Grandchild (LuckyNumber.vue)
import { inject } from 'vue';

// Inject the provided value
const maxNumber = inject('max-lucky-number');

// Use it
console.log(maxNumber.value); // 100
```

Important:

- Pass the complete ref (not `.value`) to maintain reactivity
- Key names must match exactly
- Use unique keys to avoid conflicts

9. Slots

Slots allow passing content from parent to child's template.

Basic Slot

vue

```
<!-- Child.vue -->
<template>
  <div class="child">
    <slot>Default content if nothing passed</slot>
  </div>
</template>
```

vue

```
<!-- Parent.vue -->
<template>
  <Child>
    <p>This content will appear in the slot</p>
```

```
    </Child>
  </template>
```

Named Slots

vue

```
<!-- Child.vue -->
<template>
  <div>
    <slot name="header"></slot>
    <slot></slot> <!-- Default slot -->
    <slot name="footer"></slot>
  </div>
</template>
```

vue

```
<!-- Parent.vue -->
<template>
  <Child>
    <template v-slot:header>
      <h1>Header Content</h1>
    </template>

    <p>Default slot content</p>

    <template v-slot:footer>
      <p>Footer Content</p>
    </template>
  </Child>
</template>
```

Dynamic Components

vue

```
<template>
  <component :is="currentComponent"></component>
</template>
```

```
<script setup>
import { ref, computed } from 'vue';
import LuckyNumber from './LuckyNumber.vue';
```

```
import LuckyNumberV2 from './LuckyNumberV2.vue';

const newVersion = ref(false);

const currentComponent = computed(() => {
  return newVersion.value ? LuckyNumberV2 : LuckyNumber;
});
</script>
```

Keep Alive

Preserves component state when toggling.

vue

```
<template>
  <keep-alive include="LuckyNumberV2">
    <component :is="currentComponent"></component>
  </keep-alive>
</template>
```

10. Lifecycle Hooks

javascript

```
import {
  onBeforeMount,
  onMounted,
  onBeforeUpdate,
  onUpdated,
  onBeforeUnmount,
  onUnmounted
} from 'vue';

// Before component is mounted to DOM
onBeforeMount(() => {
  console.log('App: onBeforeMount');
});

// After component is mounted to DOM
onMounted(() => {
  console.log('App: onMounted');
  // Good place for API calls
```

```
});

// Before component updates
onBeforeUpdate(() => {
  console.log('App: onBeforeUpdate');
});

// After component updates
onUpdated(() => {
  console.log('App: onUpdated');
});

// Before component is unmounted
onBeforeUnmount(() => {
  console.log('App: onBeforeUnmount');
  // Cleanup: clear intervals, event listeners
});

// After component is unmounted
onUnmounted(() => {
  console.log('App: onUnmounted');
});
```

Lifecycle Order (Parent with Child)

text

```
Parent: onBeforeMount
Child: onBeforeMount
Child: onMounted
Parent: onMounted
```

Key Takeaway: Parent doesn't finish mounting until all children are mounted.

11. Composables

Composables are reusable functions that encapsulate stateful logic.

Creating a Composable

javascript

```
// src/composables/useCounter.js
import { ref } from 'vue';

export function useCounter(initialValue = 0, step = 1) {
  const count = ref(initialValue);

  function increment() {
    count.value += step;
  }

  function decrement() {
    count.value -= step;
  }

  return {
    count,
    increment,
    decrement
  };
}
```

Using a Composable

vue

```
<template>
  <div>
    <button @click="increment">+</button>
    <span>{{ count }}</span>
    <button @click="decrement">-</button>
  </div>
</template>

<script setup>
import { useCounter } from './composables/useCounter';

// Custom initial value and step
const { count, increment, decrement } = useCounter(100, 10);
</script>
```

Benefits:

- Reusable across multiple components
- Encapsulates reactive logic

- Easier to test and maintain
- Can be nested (composable inside composable)

12. Vue Router

Installation

```
bash
```

```
npm install vue-router@4
```

Router Configuration

```
javascript
```

```
// src/router/routes.js
import { createRouter, createWebHistory } from 'vue-router';
import HomePage from '../components/home/HomePage.vue';
import Contact from '../components/home/Contact.vue';
import ProductList from '../components/product/ProductList.vue';
import ProductDetail from '../components/product/ProductDetail.vue';
import NotFound from '../components/layout/NotFound.vue';

const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: '/',
      name: 'home',
      component: HomePage
    },
    {
      path: '/contact',
      name: 'contact',
      component: Contact
    },
    {
      path: '/products',
      name: 'product-list',
      component: ProductList
    },
    {
```

```
      path: '/product/:productId',
      name: 'product-detail',
      component: ProductDetail,
      props: true
    },
    {
      path: '/login',
      name: 'login',
      component: Login
    },
    {
      path: '/*',
      name: 'not-found',
      component: NotFound
    }
  ]
});

export default router;
```

Registering Router in main.js

javascript

```
import { createApp } from 'vue';
import App from './App.vue';
import router from './router/routes';

const app = createApp(App);
app.use(router);
app.mount('#app');
```

Using Router in Components

vue

```
<!-- App.vue -->
<template>
  <Header />
  <router-view></router-view>
  <Footer />
</template>
```

Router Links

vue

```
<template>
  <nav>
    <!-- Direct path -->
    <router-link to="/">Home</router-link>

    <!-- Named route -->
    <router-link :to="{ name: 'contact' }">Contact</router-link>

    <!-- With params -->
    <router-link :to="{
      name: 'product-detail',
      params: { productId: 123, categoryId: 77 }
    }">
      Product 123
    </router-link>
  </nav>
</template>
```

Accessing Route Parameters

vue

```
<template>
  <div>
    <p>Product ID: {{ route.params.productId }}</p>
    <p v-if="route.params.categoryId">Category ID: {{ route.params.categoryId }}</p>
  </div>
</template>

<script setup>
import { useRoute } from 'vue-router';

const route = useRoute();
console.log(route.params);
</script>
```

Programmatic Navigation

javascript

```
import { useRouter } from 'vue-router';

const router = useRouter();

// Navigate to a route
router.push('/contact');
router.push({ name: 'contact' });

// Go back/forward
router.go(-1); // Back
router.go(1);  // Forward

// Replace current route
router.replace('/contact');
```

Navigation Guards

javascript

```
// Global guard
router.beforeEach((to, from, next) => {
  console.log(`Going to: ${to.name}, Coming from: ${from.name}`);

  const isAuthenticated = checkAuth();

  if (!isAuthenticated && to.name !== 'login') {
    next({ name: 'login' });
  } else {
    next();
  }
});

// Per-route guard
{
  path: '/products',
  name: 'product-list',
  component: ProductList,
  beforeEnter: (to, from, next) => {
    if (isAdmin()) {
      next();
    } else {
      next({ name: 'no-access' });
    }
  }
}
```

```
}  
}
```

Redirects

javascript

```
{  
  path: '/contact-us',  
  redirect: { name: 'contact' }  
}
```

13. API Requests with Axios

Installation

bash

```
npm install axios  
npm install -g json-server
```

Basic GET Request

vue

```
<template>  
  <div class="container">  
    <!-- Loader -->  
    <Loader v-if="destinationObj.isLoading" />  
  
    <!-- Data display -->  
    <div v-else>  
      <table class="table">  
        <thead>  
          <tr>  
            <th>Name</th>  
            <th>Days</th>  
            <th>Price</th>  
          </tr>  
        </thead>
```

```

        <tbody>
          <tr v-for="destination in destinationObj.destinations" :key="destination.id">
            <td>{{ destination.name }}</td>
            <td>{{ destination.days }}</td>
            <td>{{ destination.price }}</td>
          </tr>
        </tbody>
      </table>
    </div>
  </div>
</template>

<script setup>
import { reactive, onMounted } from 'vue';
import axios from 'axios';

const destinationObj = reactive({
  destinations: [],
  isLoading: false
});

async function loadDestinations() {
  destinationObj.isLoading = true;

  try {
    const response = await axios.get('http://localhost:3000/destinations');
    destinationObj.destinations = response.data;
  } catch (error) {
    console.error('Error fetching destinations:', error);
  } finally {
    destinationObj.isLoading = false;
  }
}

onMounted(() => {
  loadDestinations();
});
</script>

```

POST Request

javascript

```

async function addDestination(newDestination) {
  try {

```

```
const response = await axios.post(
  'http://localhost:3000/destinations',
  newDestination
);
destinationObj.destinations.push(response.data);
} catch (error) {
  console.error('Error adding destination:', error);
}
}
```

PUT/PATCH Request

javascript

```
async function updateDestination(id, updatedData) {
  try {
    const response = await axios.put(
      `http://localhost:3000/destinations/${id}`,
      updatedData
    );
    // Update local state
    const index = destinationObj.destinations.findIndex(d => d.id === id);
    if (index !== -1) {
      destinationObj.destinations[index] = response.data;
    }
  } catch (error) {
    console.error('Error updating destination:', error);
  }
}
```

DELETE Request

javascript

```
async function deleteDestination(id) {
  try {
    await axios.delete(`http://localhost:3000/destinations/${id}`);
    destinationObj.destinations = destinationObj.destinations.filter(
      d => d.id !== id
    );
  } catch (error) {
    console.error('Error deleting destination:', error);
  }
}
```

JSON Server Setup

Create data/db.json :

```
json

{
  "destinations": [
    {
      "id": 1,
      "name": "Paris",
      "days": 7,
      "funFact": "Eiffel Tower is 330 meters tall",
      "price": 1200
    },
    {
      "id": 2,
      "name": "Tokyo",
      "days": 10,
      "funFact": "Tokyo has the most Michelin stars",
      "price": 2000
    }
  ]
}
```

Run JSON Server:

```
bash

json-server --watch data/db.json
```

Fetch vs Axios

```
javascript

// Fetch (manual JSON parsing)
fetch('https://api.example.com/users')
  .then(response => response.json())
  .then(data => console.log(data));

// Axios (automatic JSON parsing)
axios.get('https://api.example.com/users')
  .then(response => console.log(response.data));
```

14. Pinia State Management

Installation

```
bash
```

```
npm install pinia
```

Setup in main.js

```
javascript
```

```
import { createApp } from 'vue';  
import { createPinia } from 'pinia';  
import App from './App.vue';
```

```
const app = createApp(App);  
const pinia = createPinia();
```

```
app.use(pinia);  
app.mount('#app');
```

Creating a Store (Options API Style)

```
javascript
```

```
// src/store/gameStore.js  
import { defineStore } from 'pinia';  
  
export const useGameStore = defineStore('gameStore', {  
  // State  
  state: () => ({  
    score: 0,  
    maxHealth: 100,  
    maxAttack: 30,  
    maxDefense: 10  
  }),  
  
  // Getters (computed)  
  getters: {  
    getScore: (state) => state.score,  
    getWinningScore: (state) => state.maxHealth,  
    getNextAttack: (state) => {
```

```

    const attack = Math.floor(Math.random() * state.maxAttack) + 1;
    console.log('Attack:', attack);
    return attack;
  },
  getNextDefense: (state) => {
    const defense = Math.floor(Math.random() * state.maxDefense) + 1;
    console.log('Defense:', defense);
    return defense;
  }
},
// Actions (methods)
actions: {
  setNextAttack() {
    const attack = Math.floor(Math.random() * this.maxAttack) + 1;
    this.score += attack;
  },
  setNextDefense() {
    const defense = Math.floor(Math.random() * this.maxDefense) + 1;
    this.score -= defense;
  },
  resetScore() {
    this.score = 50;
  }
}
});

```

Creating a Store (Composition API Style - Recommended)

javascript

```

// src/store/gameStore.js
import { defineStore } from 'pinia';
import { ref, computed } from 'vue';

export const useGameStore = defineStore('gameStore', () => {
  // State
  const score = ref(0);
  const maxHealth = ref(100);
  const maxAttack = ref(30);
  const maxDefense = ref(10);

  // Getters (computed)
  const getScore = computed(() => score.value);
  const getWinningScore = computed(() => maxHealth.value);

```

```
const getNextAttack = computed(() => {
  const attack = Math.floor(Math.random() * maxAttack.value) + 1;
  console.log('Attack:', attack);
  return attack;
});

const getNextDefense = computed(() => {
  const defense = Math.floor(Math.random() * maxDefense.value) + 1;
  console.log('Defense:', defense);
  return defense;
});

// Actions
function setNextAttack() {
  const attack = Math.floor(Math.random() * maxAttack.value) + 1;
  score.value += attack;
}

function setNextDefense() {
  const defense = Math.floor(Math.random() * maxDefense.value) + 1;
  score.value -= defense;
}

function resetScore() {
  score.value = 50;
}

// Return everything
return {
  score,
  maxHealth,
  maxAttack,
  maxDefense,
  getScore,
  getWinningScore,
  getNextAttack,
  getNextDefense,
  setNextAttack,
  setNextDefense,
  resetScore
};
});
```

Using the Store in Components

vue

```
<!-- Home.vue -->
<template>
  <div class="text-center">
    <p>Let's play the game!</p>

    <h2>Current Score: {{ gameState.score }}</h2>
    <h2>Winning Score: {{ gameState.getWinningScore }}</h2>

    <!-- Win condition -->
    <h3 v-if="gameStore.score >= gameState.maxHealth" class="text-success">
      You Win!
    </h3>

    <!-- Lose condition -->
    <h3 v-else-if="gameStore.score < 0" class="text-danger">
      You Lost!
    </h3>

    <!-- Game buttons -->
    <div v-if="gameStore.score >= 0 && gameState.score < gameState.maxHealth">
      <button @click="gameStore.setNextAttack()" class="btn btn-success">
        Increment
      </button>
      <button @click="gameStore.setNextDefense()" class="btn btn-danger">
        Decrement
      </button>
      <button @click="randomize" class="btn btn-warning">
        Random
      </button>
    </div>

    <!-- Reset button -->
    <div v-else>
      <button @click="gameStore.resetScore()" class="btn btn-primary">
        Reset Game
      </button>
    </div>
  </div>
</template>

<script setup>
import { useGameStore } from '@stores/gameStore';

const gameStore = useGameStore();
```

```
function randomize() {  
  if (Math.random() > 0.5) {  
    gameStore.setNextAttack();  
  } else {  
    gameStore.setNextDefense();  
  }  
}  
  
</script>
```

Sharing State Across Components

vue

```
<!-- Info.vue -->  
<template>  
  <div class="text-center">  
    <p>This is an info page</p>  
    <p>Score starts at 50 and you win by reaching  
      <span class="text-primary">{{ gameStore.getWinningScore }}</span>  
    </p>  
  
    <!-- Modify state from another component -->  
    <label>Update Max Health:</label>  
    <input  
      type="number"  
      v-model.number="gameStore.maxHealth"  
      class="form-control"  
    >  
  </div>  
</template>  
  
<script setup>  
import { useGameStore } from '@stores/gameStore';  
  
const gameStore = useGameStore();  
</script>
```

Key Points:

- State is shared across all components
- Changes in one component reflect in others
- No need for provide/inject or prop drilling
- Getter values are cached and reactive

- Use actions to mutate state (best practice)

Summary Cheat Sheet

Concept	Syntax	Purpose
<code>ref()</code>	<code>const x = ref(0)</code>	Reactive primitive
<code>reactive()</code>	<code>const obj = reactive({})</code>	Reactive object
<code>computed()</code>	<code>const c = computed(() => ...)</code>	Cached derived value
<code>watch()</code>	<code>watch(source, callback)</code>	Side effects on change
<code>v-model</code>	<code><input v-model="x"></code>	Two-way binding
<code>v-bind / :</code>	<code>:href="url"</code>	Bind attribute
<code>v-on / @</code>	<code>@click="fn"</code>	Listen to event
<code>v-if</code>	<code>v-if="condition"</code>	Conditional render
<code>v-for</code>	<code>v-for="item in items"</code>	List rendering
<code>v-show</code>	<code>v-show="condition"</code>	Toggle visibility
Props	<code>defineProps(['name'])</code>	Parent to child
Emits	<code>defineEmits(['event'])</code>	Child to parent
Provide	<code>provide('key', value)</code>	Avoid prop drilling
Inject	<code>inject('key')</code>	Receive provided value
Slot	<code><slot></slot></code>	Content distribution
Lifecycle	<code>onMounted(() => ...)</code>	Component lifecycle
Composable	<code>useCounter()</code>	Reusable logic
Router	<code>useRouter()</code>	Navigation

Concept	Syntax	Purpose
Route	<code>useRoute()</code>	Route params
Pinia Store	<code>useGameStore()</code>	State management

explain everything again deeply explain the code also

Vue.js Complete Course — Deep Dive with Code Explanations

I'll walk through every concept again, but this time I'll explain **what each line of code does, why it exists, and how it works under the hood.**

Table of Contents

- 1. Vue Fundamentals — The Absolute Basics
- 2. Reactivity: ref vs reactive
- 3. Vue Directives Explained
- 4. Event Handling
- 5. v-model & Forms
- 6. Computed Properties & Watchers
- 7. Components Deep Dive
- 8. Component Communication
- 9. Slots
- 10. Lifecycle Hooks
- 11. Composables
- 12. Vue Router
- 13. API Requests
- 14. Pinia

1. Vue Fundamentals

The Minimal Vue App

html

```
<!DOCTYPE html>
<html>
<head>
  <!-- Step 1: Load Vue from CDN -->
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
  <!-- Step 2: Create a mount point -->
  <div id="app">
    <h1>{{ message }}</h1>
  </div>

  <script>
    // Step 3: Destructure what we need from the global Vue object
    const { createApp, ref } = Vue;

    // Step 4: Create the app
    const app = createApp({
      // setup() is the entry point for Composition API
      setup() {
        const message = ref('Hello Vue 3');
        return { message }; // MUST return to expose to template
      }
    });

    // Step 5: Mount to the DOM element
    app.mount('#app');
  </script>
</body>
</html>
```

Line-by-line explanation:

- `<script src="...">` — Vue is loaded as a global object called `vue`. In a real project (Vite), we'd `import` instead.
- `<div id="app">` — Vue needs a "root" DOM element to control. Everything inside this `div` is managed by Vue.
- `{{ message }}` — This is **mustache syntax** (double curly braces). Vue replaces it with the value of `message`. It's not HTML — it's a Vue template expression.

- `createApp({...})` — Creates a Vue application instance. The object passed in is the **root component options**.
- `setup()` — The entry function for the Composition API. Everything reactive lives here. It runs **before** the component is created.
- `ref('Hello Vue 3')` — Creates a reactive reference. More on this below.
- `return { message }` — If you don't return it, the template can't see it. This is one of the most common beginner mistakes. Vue will warn: *"Property 'message' was accessed during render but is not defined."*
- `app.mount('#app')` — Tells Vue: "Take control of the `#app` element and render the template inside it."

Why We Mount

When you call `app.mount('#app')` :

1. Vue finds the DOM element with `id="app"` .
2. It reads the inner HTML as a **template**.
3. It compiles that template into a **render function**.
4. It executes the render function, which produces a **Virtual DOM**.
5. It replaces the real DOM content with what the Virtual DOM describes.
6. From now on, whenever reactive data changes, it re-runs the render function, diffs the new Virtual DOM against the old one, and patches only the changed parts.

2. Reactivity

Why Reactivity Matters

In plain JavaScript:

```
javascript
```

```
let count = 0;
document.getElementById('output').textContent = count;

count = 5; // DOM does NOT update automatically
```

You'd have to manually update the DOM every time. Vue's reactivity system solves this — it **tracks** which pieces of data are used where, and when they change, it knows exactly what to update.

ref() — For Primitives

javascript

```
import { ref } from 'vue';

const count = ref(0);           // Creates a reactive ref
console.log(count);             // RefImpl { value: 0 }
console.log(count.value);       // 0 – access the actual value
count.value = 5;                // Update it
console.log(count.value);       // 5
```

What's happening internally:

- `ref(0)` creates a **wrapper object** with a `.value` property.
- Vue uses JavaScript's `Proxy` (or `Object.defineProperty` in older versions) to intercept reads and writes to `.value`.
- When you **read** `.value`, Vue registers the current "effect" (like a component render or a computed) as a **dependency**.
- When you **write** `.value`, Vue triggers all registered dependencies to re-run.

That's why you need `.value` — it's a special getter/setter that lets Vue know when you're reading or writing.

In templates, you don't need `.value`:

html

```
<h1>{{ count }}</h1> <!-- Vue unwraps it automatically -->
```

Vue automatically unwraps top-level refs in templates. Inside `setup()`, you use `.value`.

reactive() — For Objects

javascript

```
import { reactive } from 'vue';

const employee = reactive({
  name: 'John',
  age: 30,
  salary: 50000
});

employee.age = 31; // No .value needed - direct mutation
console.log(employee.age); // 31
```

What's happening internally:

- `reactive()` returns a **Proxy** that wraps the original object.
- Any property access (`employee.age`) goes through the Proxy's `get` trap.
- Any property assignment (`employee.age = 31`) goes through the Proxy's `set` trap.
- Vue registers dependencies on read and triggers updates on write.

ref VS reactive — The Rules

Aspect	ref	reactive
Works with	Primitives AND objects	Objects/arrays only
Access	<code>.value</code> (in script)	Direct
Template	Auto-unwrapped	Auto-unwrapped
Reassignable	Yes (<code>count.value = 10</code>)	No (can't replace whole object)
TypeScript	Better	Slightly worse

Why `reactive` can't be reassigned:

javascript

```
let state = reactive({ count: 0 });
state = reactive({ count: 5 }); // You've lost the original proxy!
// Any template still referencing the old proxy won't update.
```

`ref` doesn't have this problem because you reassign `.value`, and the wrapper object stays the same.

Behind the Scenes: `ref` Calls `reactive`

javascript

```
function ref(value) {
  return {
    get value() {
      track(this, 'value'); // Register dependency
      return value;
    },
    set value(newValue) {
      value = newValue;
      trigger(this, 'value'); // Trigger updates
    }
  };
}
```

(Simplified — the real implementation handles objects by wrapping them in `reactive`.)

3. Vue Directives

Directives are special attributes starting with `v-`. Vue recognizes them during template compilation and gives them special behavior.

`v-html` — Render Raw HTML

html

```
<template>
  <span v-html="getMessage()"></span>
</template>

<script setup>
const message = 'Hello Vue';

function getMessage() {
  return `

# <u>${message}</u></h1>`; }


```

```
}
</script>
```

Explanation:

- Without `v-html`, `{{ getMessage() }}` would display the literal string `<h1><u>Hello Vue</u></h1>` (escaped).
- `v-html` tells Vue: "Parse this as HTML, not text."
- **Security warning:** Never use `v-html` with user-provided content — it can lead to XSS attacks.

v-once — Render Once

```
html

<p v-once>Render once: {{ luckyNumber }}</p>
<p>Regular: {{ luckyNumber }}</p>
```

Explanation:

- `v-once` tells Vue: "Render this element once, then never update it again."
- The value is captured at first render and stays frozen.
- Useful for static content that never changes (optimization).

v-bind — Bind Attributes

```
html

<!-- Full syntax -->
<a v-bind:href="blogUrl">Visit Blog</a>

<!-- Shorthand (the colon) -->
<a :href="blogUrl">Visit Blog</a>
```

Explanation:

- Without `v-bind`, `href="blogUrl"` would literally set the attribute to the string `"blogUrl"`.
- With `v-bind`, Vue evaluates `blogUrl` as a JavaScript expression and uses its value.

- The shorthand `:` is identical to `v-bind:` — just less typing.

Class binding with objects:

html

```
<div :class="{ active: isActive, 'text-danger': hasError }"></div>
```

Explanation:

- The object keys are class names.
- The values are booleans determining whether the class is added.
- `active` is added only if `isActive` is `true`.
- `'text-danger'` is quoted because it contains a hyphen.

Style binding:

html

```
<div :style="{ backgroundColor: bgColor, fontSize: fontSize + 'px' }"></div>
```

Explanation:

- Keys are camelCase CSS properties.
- Values are strings (must include units).
- Hyphenated keys can be quoted: `'background-color'`.

v-if / v-else-if / v-else

html

```
<p v-if="employee.hasCorporateCard">
  Spending limit: {{ employee.spendingLimit }}
</p>
<p v-else-if="employee.salary > 25000 && employee.yearsWithCompany > 1">
  Eligible for corporate card
</p>
<p v-else>
  Not eligible
</p>
```

Explanation:

- `v-if` completely **removes** the element from the DOM when false.
- Vue evaluates conditions in order. The first true one wins.
- `v-else-if` and `v-else` must be immediate siblings of the `v-if` element.
- `&&` is JavaScript logical AND — both conditions must be true.

v-show

html

```
<p v-show="employee.hasCorporateCard">Spending limit</p>
```

Explanation:

- `v-show` **keeps** the element in the DOM but adds `display: none` via inline CSS.
- The element is always rendered — just hidden.

When to use which:

- `v-if` — When the condition rarely changes (e.g., user role, feature flags). Saves initial render cost.
- `v-show` — When the condition toggles frequently (e.g., dropdown open/closed). No re-render cost.

v-for

html

```
<ul>
  <li v-for="(luckyNumber, index) in luckyNumbers" :key="index">
    {{ index + 1 }}. {{ luckyNumber }}
  </li>
</ul>
```

Explanation:

- `v-for` iterates over an array (or object).
- First argument is the item, second is the index.

- `:key="index"` gives Vue a unique identifier for each item — crucial for efficient DOM diffing.

Object iteration:

html

```
<li v-for="(value, key, index) in employee" :key="key">
  {{ key }}: {{ value }}
</li>
```

- For objects, the order is `(value, key, index)`.
- Note: value comes **first**, then key.

Why `:key` matters:

When Vue re-renders a list, it uses the `key` to identify which items are the same. Without a key, Vue uses the index, which can cause weird behavior when items are reordered or removed.

html

```
<!-- Bad: input values don't follow their projects when shuffled -->
<div v-for="project in employee.projects">
  <span>{{ project.project }}</span>
  <input type="text">
</div>

<!-- Good: input values stay with their projects -->
<div v-for="project in employee.projects" :key="project.project">
  <span>{{ project.project }}</span>
  <input type="text">
</div>
```

4. Event Handling

v-on / @

html

```
<!-- Full syntax -->
<button v-on:click="handleClick">Click Me</button>

<!-- Shorthand -->
<button @click="handleClick">Click Me</button>
```

Explanation:

- `v-on:click` attaches a click event listener.
- The value is a **method name** or an **inline expression**.
- When the method name is given, Vue passes the event object automatically.
- `@click` is shorthand for `v-on:click`.

Passing arguments:

html

```
<button @click="handleClick('Hello Vue')">Click Me</button>
```

- Now Vue calls `handleClick('Hello Vue')` — the event object is NOT passed automatically.
- To get both, use `$event` :

html

```
<button @click="handleClick('Hello Vue', $event)">Click Me</button>
```

Event Modifiers

html

```
<button @click.stop="handleClick">Stop propagation</button>
<form @submit.prevent="handleSubmit">Prevent default</form>
<button @click.once="handleClick">Only once</button>
```

Explanation:

- `.stop` calls `event.stopPropagation()` — prevents bubbling to parent.
- `.prevent` calls `event.preventDefault()` — prevents default browser behavior.

- `.once` removes the listener after the first trigger.

Key modifiers:

html

```
<input @keyup.enter="handleEnter">
<input @keyup.ctrl.enter="handleCtrlEnter">
```

- `.enter` — only triggers on Enter key.
- `.ctrl.enter` — only triggers on Ctrl+Enter.
- Vue supports many key aliases: `.enter`, `.tab`, `.delete`, `.esc`, `.space`, `.up`, `.down`, `.left`, `.right`.

The Event Object

javascript

```
function handleClick(event) {
  console.log(event.type);      // "click"
  console.log(event.clientX);   // X coordinate
  console.log(event.clientY);   // Y coordinate
  console.log(event.target);    // The clicked element
}
```

Every DOM event provides this object with useful info.

5. v-model & Forms

Two-Way Binding

html

```
<template>
  <input v-model="name">
  <p>Name: {{ name }}</p>
</template>
```

```
<script setup>
import { ref } from 'vue';
```

```
const name = ref('Ben');  
</script>
```

What `v-model` does:

It's syntactic sugar for:

```
html  
  
<input  
  :value="name"  
  @input="name = $event.target.value"  
>
```

So:

1. `:value="name"` — The input's value is set from `name`.
2. `@input="name = $event.target.value"` — When the user types, `name` is updated.

This is **two-way binding**: data → view AND view → data.

On Different Input Types

```
html  
  
<!-- Checkbox: binds to boolean -->  
<input type="checkbox" v-model="subscribe">  
  
<!-- Radio: binds to the value of the selected radio -->  
<input type="radio" v-model="gender" value="male"> Male  
<input type="radio" v-model="gender" value="female"> Female  
  
<!-- Select: binds to selected option's value -->  
<select v-model="gender">  
  <option value="male">Male</option>  
  <option value="female">Female</option>  
</select>  
  
<!-- Number: .number converts string to number -->  
<input type="number" v-model.number="age">
```

Model Modifiers

html

```
<input v-model.trim="name">
<input v-model.lazy="age">
<input v-model.number="phone">
```

- `.trim` — Trims leading/trailing whitespace on every update.
- `.lazy` — Updates on `change` (blur) instead of `input` (every keystroke).
- `.number` — Converts the string value to a number.

Form Validation Example — Deep Explanation

vue

```
<template>
  <form @submit.prevent="handleSubmit">
    <div class="mb-3">
      <label>Name:</label>
      <input
        v-model.trim="formObj.name"
        @blur="validateName"
        class="form-control"
      >
      <div v-if="!formObj.isNameValid" class="text-danger">
        Name is required (min 3 characters)
      </div>
    </div>

    <div class="mb-3">
      <label>Email:</label>
      <input v-model.trim="formObj.email" class="form-control">
    </div>

    <div class="mb-3">
      <label>Phone:</label>
      <input v-model.number="formObj.phone" class="form-control">
    </div>

    <div class="form-check">
      <input type="checkbox" v-model="formObj.subscribe" class="form-check-input">
      <label class="form-check-label">Subscribe</label>
    </div>

    <div v-if="errorList.length > 0" class="mt-3">
```

```
<ul class="text-danger">
  <li v-for="error in errorList" :key="error">{{ error }}</li>
</ul>
</div>

<button type="submit" class="btn btn-primary mt-3">Submit</button>
</form>
</template>

<script setup>
import { reactive } from 'vue';

const formObj = reactive({
  name: '',
  email: '',
  phone: '',
  subscribe: false,
  isNameValid: true
});

const errorList = reactive([]);

function validateName() {
  formObj.isNameValid = formObj.name.length >= 3;
}

function handleSubmit() {
  errorList.length = 0; // Clear existing errors

  if (formObj.name.length < 3) {
    errorList.push('Name must be at least 3 characters');
  }
  if (!formObj.email) {
    errorList.push('Email is required');
  }
  if (!formObj.subscribe) {
    errorList.push('Please subscribe to the newsletter');
  }

  if (errorList.length === 0) {
    alert('Form submitted successfully!');
    console.log('Form data:', formObj);
  }
}
</script>
```

Line-by-line explanation:

- `@submit.prevent` — The form's submit event is intercepted; default page reload is prevented.
- `v-model.trim="formObj.name"` — Two-way binding with automatic trimming.
- `@blur="validateName"` — When focus leaves the input, validate.
- `v-if="!formObj.isNameValid"` — Show error only when validation fails.
- `v-model.number="formObj.phone"` — Phone is stored as a number, not a string.
- `errorList.length = 0` — Clearing an array reactively (assigning to `.length` works because `reactive` tracks it).
- `errorList.push(...)` — Pushing errors triggers reactivity.
- `if (errorList.length === 0)` — Only submit if no errors.

6. Computed & Watchers

Computed Properties

javascript

```
import { ref, computed } from 'vue';

const firstName = ref('Ben');
const lastName = ref('Patel');

const fullName = computed(() => {
  console.log('Computing full name...');
  return `${firstName.value} ${lastName.value}`;
});
```

Why computed?

- `fullName` is **cached**. It only recalculates when `firstName` or `lastName` change.
- If some unrelated reactive value changes (e.g., `counter`), `fullName` does NOT recalculate.
- A regular function would recalculate on every render.

How caching works:

1. When you first access `fullName.value`, Vue runs the function and records the dependencies (`firstName`, `lastName`).
2. It caches the result.
3. Next time you access it, if dependencies haven't changed, it returns the cached value.
4. If a dependency changes, the cache is invalidated, and the next access recalculates.

Watchers

javascript

```
import { ref, watch } from 'vue';

const counter = ref(0);

watch(counter, (newValue, oldValue) => {
  console.log(`Counter changed from ${oldValue} to ${newValue}`);
  if (newValue === 5) {
    alert('Counter reached 5!');
  }
});
```

Explanation:

- `watch(source, callback)` — `source` is the reactive value to watch.
- The callback receives `(newValue, oldValue)`.
- Runs **whenever** the source changes.
- Unlike `computed`, watchers are for **side effects** (alerts, API calls, logging), not for deriving values.

Watching a reactive object property:

javascript

```
const obj = reactive({ count: 0 });

watch(
  () => obj.count, // Must use a getter
  (newValue, oldValue) => { /* ... */ }
);
```

Why the getter? Because `obj.count` is a primitive — you'd be passing the value, not the source. The getter tells Vue "watch this specific property."

Async watchers:

```
javascript

watch(counter, async (newValue) => {
  if (newValue === 5) {
    await new Promise(resolve => setTimeout(resolve, 3000));
    console.log('Async operation completed');
  }
});
```

Watchers can be async — computed properties cannot.

Computed vs Watchers — Deep Comparison

Aspect	Computed	Watcher
Returns value	Yes	No
Cached	Yes	No
Async	No	Yes
Use case	Derived state	Side effects
Re-evaluates	Only when deps change	On every change

7. Components

Creating a Vue Project with Vite

```
bash

npm create vue@latest
```

You'll be asked:

- Project name
- Add TypeScript? (No for this course)
- Add JSX support? (No)
- Add Vue Router? (Optional)
- Add Pinia? (Optional)
- Add Vitest? (No)
- Add ESLint? (No)

Then:

```
bash
```

```
cd project-name  
npm install  
npm run dev
```

What Vite does:

- Compiles .vue files into JavaScript.
- Provides a dev server with Hot Module Replacement (HMR).
- Bundles your app for production.

Anatomy of a .vue File

```
vue
```

```
<template>  
  <div>  
    <h1>{{ message }}</h1>  
  </div>  
</template>  
  
<script setup>  
import { ref } from 'vue';  
const message = ref('Hello Vue 3');  
</script>  
  
<style scoped>  
h1 {  
  color: green;
```

```
}
</style>
```

Three blocks:

- `<template>` — The HTML structure. Must have one root element (or use `<template>` wrapper).
- `<script setup>` — Composition API code. Everything defined here is automatically available in the template.
- `<style scoped>` — CSS. `scoped` means it only applies to this component.

Why `<script setup>` ?

- No `export default`, no `setup()` function, no explicit `return` needed.
- Vue's compiler transforms it automatically.
- Cleaner, more concise.

main.js — The Entry Point

javascript

```
import { createApp } from 'vue';
import App from './App.vue';

const app = createApp(App);
app.mount('#app');
```

Explanation:

- `createApp(App)` — Creates the app with `App.vue` as the root component.
- `app.mount('#app')` — Attaches to the DOM element with `id="app"` (found in `index.html`).

index.html

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Vue App</title>
```

```
</head>
<body>
  <div id="app"></div>
  <script type="module" src="/src/main.js"></script>
</body>
</html>
```

- The `<div id="app">` is the mount point.
- `<script type="module" src="/src/main.js">` loads the entry file.

Component Registration

Global (in `main.js`):

javascript

```
import ButtonCounter from './components/ButtonCounter.vue';
app.component('button-counter', ButtonCounter);
```

- Available in **every** component.
- Increases initial bundle size.

Local (recommended):

vue

```
<script setup>
import ButtonCounter from './components/ButtonCounter.vue';
</script>
```

- Only available in the importing component.
- Better for code-splitting and performance.

8. Component Communication

Props — Parent to Child

Parent:

vue

```
<Contact
  name="Ben"
  :phone="1234567890"
  :owner-name="contactOwner"
/>
```

Explanation:

- `name="Ben"` — Static string prop.
- `:phone="1234567890"` — Dynamic binding (number).
- `:owner-name="contactOwner"` — Kebab-case in template → camelCase in script.

Child:

vue

```
<script setup>
const props = defineProps({
  name: {
    type: String,
    required: true
  },
  phone: {
    type: Number,
    required: true
  },
  ownerName: {
    type: String,
    default: 'Not Applicable'
  }
});
</script>
```

Prop validation:

- `type` — Expected type (String, Number, Boolean, Array, Object, Function).
- `required` — Must be provided.
- `default` — Fallback if not provided.

Why camelCase?

- HTML attributes are case-insensitive.
- Vue convention: kebab-case in templates, camelCase in script.
- `:owner-name` → `ownerName` .

Props are read-only:

javascript

```
props.name = 'New'; // ERROR: Set operation on key "name" failed: target is readonly
```

Only the **parent** owns prop data. The child must **emit** changes.

Emits — Child to Parent

Child:

vue

```
<script setup>
const props = defineProps(['isFavorite']);
const emit = defineEmits(['update-favorite']);

function toggleFavorite() {
  emit('update-favorite', {
    isFavorite: props.isFavorite,
    name: props.name
  });
}
</script>
```

Explanation:

- `defineEmits(['update-favorite'])` declares the event the child can emit.
- `emit('update-favorite', payload)` fires the event with data.

Parent:

vue

```
<Contact
  :is-favorite="contact.isFavorite"
  @update-favorite="handleUpdateFavorite"
/>
```

```
javascript
```

```
function handleUpdateFavorite(eventData) {
  console.log(eventData); // { isFavorite: true, name: 'Ben' }
  contact.isFavorite = !eventData.isFavorite;
}
```

Flow:

1. Child clicks button → calls `toggleFavorite()` .
2. Child emits `update-favorite` with payload.
3. Parent's `@update-favorite` listener fires.
4. Parent updates its own data.
5. New data flows back down via props.

Passing Methods as Props

Instead of emitting, you can pass a **parent method as a prop**:

Parent:

```
vue
```

```
<AddContact :on-add-contact="handleAddContact" />
```

Child:

```
vue
```

```
<script setup>
const props = defineProps({
  onAddContact: {
    type: Function,
    required: true
  }
});

function addContact() {
  props.onAddContact({ name: 'Test', phone: 123 });
}
</script>
```

Both work. Emits are more idiomatic Vue; passing methods is more React-like.

Provide/Inject — Avoiding Prop Drilling

Problem: If a grandchild needs data from a grandparent, you'd have to pass it through the child (which doesn't use it).

Solution:

Grandparent:

javascript

```
import { provide, ref } from 'vue';
const maxNumber = ref(100);
provide('max-lucky-number', maxNumber);
```

Grandchild:

javascript

```
import { inject } from 'vue';
const maxNumber = inject('max-lucky-number');
console.log(maxNumber.value); // 100
```

Key points:

- Provide the **ref itself**, not `.value` — this preserves reactivity.
- Keys must match exactly.
- Use unique keys to avoid conflicts.

9. Slots

Basic Slot

Child:

vue

```
<template>
  <div>
    <slot>Fallback content</slot>
  </div>
</template>
```

Parent:

vue

```
<Child>
  <p>This goes into the slot</p>
</Child>
```

The `<p>` is rendered inside the child where `<slot>` is.

Named Slots

Child:

vue

```
<slot name="header"></slot>
<slot></slot> <!-- default -->
<slot name="footer"></slot>
```

Parent:

vue

```
<template v-slot:header>
  <h1>Header</h1>
</template>
```

```
<p>Default content</p>
```

```
<template v-slot:footer>
  <p>Footer</p>
</template>
```

Slots Belong to the Parent

Even though slot content renders inside the child, it **belongs to the parent**. Any data used in the slot comes from the parent's scope.

Dynamic Components

vue

```
<template>
  <component :is="currentComponent"></component>
</template>

<script setup>
import { ref, computed } from 'vue';
import LuckyNumber from './LuckyNumber.vue';
import LuckyNumberV2 from './LuckyNumberV2.vue';

const newVersion = ref(false);
const currentComponent = computed(() =>
  newVersion.value ? LuckyNumberV2 : LuckyNumber
);
</script>
```

Explanation:

- `<component :is="...">` is a special built-in component.
- `:is` accepts a component definition (imported component).
- When the value changes, Vue swaps the component.

Keep Alive

vue

```
<keep-alive include="LuckyNumberV2">
  <component :is="currentComponent"></component>
</keep-alive>
```

Why? By default, when a dynamic component is swapped out, it's destroyed. `keep-alive` preserves its state.

- `include="ComponentName"` — Only cache these.
- `exclude` — Cache everything except these.

- Multiple: `include="['A', 'B']"` (bind with `:include`).

10. Lifecycle Hooks

javascript

```
import {
  onBeforeMount, onMounted,
  onBeforeUpdate, onUpdated,
  onBeforeUnmount, onUnmounted
} from 'vue';

onBeforeMount(() => {
  console.log('About to mount');
});

onMounted(() => {
  console.log('Mounted');
  // Perfect for API calls
});

onBeforeUpdate(() => {
  console.log('About to update');
});

onUpdated(() => {
  console.log('Updated');
});

onBeforeUnmount(() => {
  console.log('About to unmount');
  // Clean up timers, listeners
});

onUnmounted(() => {
  console.log('Unmounted');
});
```

Order with Parent + Child

text

```
Parent: onBeforeMount
Child: onBeforeMount
Child: onMounted
Parent: onMounted
```

Why this order? The parent can't be "mounted" until all its children are ready.

When updating:

```
text
```

```
Parent: onBeforeUpdate
Child: onBeforeUpdate
Child: onUpdated
Parent: onUpdated
```

When unmounting:

```
text
```

```
Parent: onBeforeUnmount
Child: onBeforeUnmount
Child: onUnmounted
Parent: onUnmounted
```

11. Composables

```
javascript
```

```
// composables/useCounter.js
import { ref } from 'vue';

export function useCounter(initialValue = 0, step = 1) {
  const count = ref(initialValue);

  function increment() {
    count.value += step;
  }

  function decrement() {
    count.value -= step;
  }
}
```

```
    return { count, increment, decrement };  
  }
```

Usage:

vue

```
<script setup>  
import { useCounter } from '@/composables/useCounter';  
const { count, increment, decrement } = useCounter(100, 10);  
</script>
```

Why composables?

- Extract reusable logic.
- Each call creates **fresh state** — no shared state like mixins.
- Can be nested (composable using another composable).
- Naming convention: start with `use` .

12. Vue Router

Installation

bash

```
npm install vue-router@4
```

Route Configuration

javascript

```
// router/routes.js  
import { createRouter, createWebHistory } from 'vue-router';  
import HomePage from '../components/home/HomePage.vue';  
import Contact from '../components/home/Contact.vue';  
  
const router = createRouter({  
  history: createWebHistory(),  
  routes: [  

```

```

{ path: '/', name: 'home', component: HomePage },
{ path: '/contact', name: 'contact', component: Contact },
{
  path: '/product/:productId',
  name: 'product-detail',
  component: ProductDetail,
  props: true // Pass params as props
},
{
  path: '/*',
  name: 'not-found',
  component: NotFound
}
]
});

export default router;

```

Explanation:

- `createWebHistory()` — Uses HTML5 history mode (clean URLs, no #).
- `routes` — Array of route objects.
- `path` — URL pattern.
- `:productId` — Dynamic segment (accessed via `route.params.productId`).
- `:catchAll(*)` — Catches any unmatched route → 404.
- `props: true` — Passes route params as component props.

Registering

```

javascript

// main.js
import router from './router/routes';
app.use(router);

```

Using in Template

```

vue

<router-link to="/">Home</router-link>
<router-link :to="{ name: 'contact' }">Contact</router-link>
<router-link :to="{

```

```

    name: 'product-detail',
    params: { productId: 123 }
  }">Product 123</router-link>

<router-view></router-view>

```

Explanation:

- `<router-link>` — Renders as `<a>` but prevents full page reload.
- `to="/"` — Direct path.
- `to="{ name: 'contact' }"` — Named route.
- `to="{ name: '...', params: {...} }"` — With params.
- `<router-view>` — Where the matched component is rendered.

Accessing Route Data

javascript

```

import { useRoute } from 'vue-router';
const route = useRoute();
console.log(route.params.productId);

```

Navigation

javascript

```

import { useRouter } from 'vue-router';
const router = useRouter();

router.push('/contact');
router.push({ name: 'contact' });
router.go(-1); // Back
router.go(1);  // Forward

```

Navigation Guards

javascript

```

router.beforeEach((to, from, next) => {
  const isAuthenticated = checkAuth();
  if (!isAuthenticated && to.name !== 'login') {

```

```
      next({ name: 'login' });  
    } else {  
      next();  
    }  
  });
```

Explanation:

- `to` — Route being navigated to.
- `from` — Current route.
- `next()` — Continue navigation.
- `next({ name: 'login' })` — Redirect.

Per-Route Guard

javascript

```
{  
  path: '/products',  
  beforeEnter: (to, from, next) => {  
    if (isAdmin()) next();  
    else next({ name: 'no-access' });  
  }  
}
```

13. API Requests

Installation

bash

```
npm install axios
```

GET Request

javascript

```
import axios from 'axios';
```

```
const destinationObj = reactive({
  destinations: [],
  isLoading: false
});

async function loadDestinations() {
  destinationObj.isLoading = true;
  try {
    const response = await axios.get('http://localhost:3000/destinations');
    destinationObj.destinations = response.data;
  } catch (error) {
    console.error('Error:', error);
  } finally {
    destinationObj.isLoading = false;
  }
}

onMounted(() => {
  loadDestinations();
});
```

Explanation:

- `axios.get(url)` — Returns a Promise.
- `await` — Pauses until resolved.
- `response.data` — The parsed JSON body.
- `try/catch/finally` — Handle errors and always clean up loading state.
- `onMounted` — Call when component mounts.

POST

javascript

```
async function addDestination(newDest) {
  const response = await axios.post(
    'http://localhost:3000/destinations',
    newDest
  );
  destinationObj.destinations.push(response.data);
}
```

PUT

javascript

```
async function updateDestination(id, data) {
  const response = await axios.put(
    `http://localhost:3000/destinations/${id}`,
    data
  );
  const index = destinationObj.destinations.findIndex(d => d.id === id);
  if (index !== -1) destinationObj.destinations[index] = response.data;
}
```

DELETE

javascript

```
async function deleteDestination(id) {
  await axios.delete(`http://localhost:3000/destinations/${id}`);
  destinationObj.destinations = destinationObj.destinations.filter(
    d => d.id !== id
  );
}
```

JSON Server

bash

```
json-server --watch data/db.json
```

Creates a REST API from a JSON file. Great for prototyping.

Axios vs Fetch

javascript

```
// Fetch: manual JSON parsing
fetch(url)
  .then(res => res.json())
  .then(data => console.log(data));

// Axios: automatic JSON parsing
axios.get(url)
  .then(res => console.log(res.data));
```

Axios is more concise and has better error handling.

14. Pinia

Installation

bash

```
npm install pinia
```

Setup

javascript

```
import { createPinia } from 'pinia';  
const pinia = createPinia();  
app.use(pinia);
```

Store (Composition API Style)

javascript

```
import { defineStore } from 'pinia';  
import { ref, computed } from 'vue';  
  
export const useGameStore = defineStore('gameStore', () => {  
  // State  
  const score = ref(0);  
  const maxHealth = ref(100);  
  const maxAttack = ref(30);  
  const maxDefense = ref(10);  
  
  // Getters (computed)  
  const getScore = computed(() => score.value);  
  const getWinningScore = computed(() => maxHealth.value);  
  
  // Actions  
  function setNextAttack() {  
    const attack = Math.floor(Math.random() * maxAttack.value) + 1;  
    score.value += attack;  
  }  
}
```

```

function setNextDefense() {
  const defense = Math.floor(Math.random() * maxDefense.value) + 1;
  score.value -= defense;
}

function resetScore() {
  score.value = 50;
}

return {
  score, maxHealth, maxAttack, maxDefense,
  getScore, getWinningScore,
  setNextAttack, setNextDefense, resetScore
};
});

```

Explanation:

- `defineStore('gameStore', () => {...})` — First arg is a unique ID; second is a setup function.
- State is just `ref s`.
- Getters are `computed s`.
- Actions are regular functions.
- Everything is returned to expose it.

Using the Store

vue

```

<template>
  <h2>Score: {{ gameStore.score }}</h2>
  <h2>Winning Score: {{ gameStore.getWinningScore }}</h2>
  <button @click="gameStore.setNextAttack()">Attack</button>
</template>

<script setup>
import { useGameStore } from '@stores/gameStore';
const gameStore = useGameStore();
</script>

```

Explanation:

- `useGameStore()` returns the store instance.
- Access state directly: `gameStore.score` .
- Access getters directly: `gameStore.getWinningScore` .
- Call actions: `gameStore.setNextAttack()` .

Shared State

Any component that calls `useGameStore()` gets the **same** store instance. Changes in one component are visible in all others — no prop drilling, no provide/inject.

Key Takeaways

1. **Reactivity** is Vue's superpower — `ref` for primitives, `reactive` for objects.
2. **Directives** (`v-if` , `v-for` , `v-bind` , `v-on` , `v-model`) are how you connect logic to templates.
3. **Computed** is for derived values (cached); **watchers** are for side effects.
4. **Components** are the building blocks — props down, emits up.
5. **Composables** let you reuse stateful logic.
6. **Vue Router** handles navigation; **navigation guards** handle access control.
7. **Axios** makes API calls clean and simple.
8. **Pinia** is the modern state management solution — cleaner than Vuex, works beautifully with Composition API.

Would you like me to go deeper on any specific topic — like the reactivity internals, the Virtual DOM diffing algorithm, or a full project walkthrough?